PDF 1 : INTRODUCCION

1. Definición Formal y Ciclo de Vida de la Seguridad
La seguridad se define fundamentalmente como previsibilidad: asegurar que los sistemas, procesos y entornos se comporten exactamente como se espera. Para lograr esto, la seguridad debe integrarse en todas las fases del ciclo de vida:

Planificación (Planning):El objetivo es diseñar una solución que cumpla requisitos bajo un contexto normativo.

Criterio de éxito: No debe haber fallos lógicos. Todos los estados de operación deben ser los predichos; no pueden existir estados adicionales que escapen a la lógica esperada. Debe seguir normas específicas del sector (ej. ISO 27001).

Desarrollo (Development):
Implementar el diseño sin introducir nuevos modos de operación no deseados.
Criterio de éxito: Sin bugs, sin crashes, sin fugas de información y con un consumo de recursos adecuado.Requiere pruebas (tests) rigurosas para asegurar que hace solo lo que se espera.
Ejecución (Execution):
El código se ejecuta tal como fue escrito.
El entorno debe estar controlado para evitar comportamientos anómalos introducidos por hardware (velocidad de disco, RAM) o comunicaciones no confiables.
Personas y Socios (People):

El comportamiento del personal no debe impactar negativamente.

Se mitiga con normas, entrenamiento para distinguir comportamientos correctos/incorrectos e incentivos adecuados.

2. Objetivos de Seguridad: CIA + Privacidad
Los tres pilares clásicos (CIA) se detallan con medidas específicas que debes memorizar:

Confidencialidad (Confidentiality): La información solo es accesible por un grupo restringido.
Medidas: Cifrado, contraseñas fuertes, gestión de identidad, control físico (puertas, guardias).
Integridad (Integrity): La información permanece inalterada.

Medidas: Hashes (control de integridad), copias de seguridad (backups), controles de acceso, verificación de datos.

Disponibilidad (Availability): La información está disponible para quien la necesita.

Medidas: Redundancia, planes de recuperación de desastres (DRP), virtualización, monitorización.
Extra: Privacidad (Privacy): Se refiere a cómo se maneja la información personal (adquisición, procesamiento, almacenamiento, borrado).

Medidas: Procesos transparentes, cifrado, logs y controles de autenticidad.

3. ¿Contra qué nos defendemos? (Threat Landscape)
El documento clasifica los objetivos de defensa en tres niveles:

Eventos Catastróficos: Fenómenos naturales (inundaciones, rayos) o degradación física del hardware (fallos de disco, errores de bit en RAM).

Fallos Ordinarios: Cortes de energía, fallos internos del sistema (kernel panics, pantallazos azules), deadlocks o bugs de software.

Adversarios (Actividades no autorizadas): Iniciadas por alguien "de fuera" o "de dentro". Incluye robo de info, vandalismo, uso de recursos (minería de cripto, por ejemplo) y Denegación de Servicio (DoS).

4. Arquitectura de Seguridad: Conceptos Jerárquicos
Es vital entender la relación entre estos cuatro términos:

Dominios de Seguridad: Conjunto de entidades que comparten atributos de seguridad similares.Permiten gestionar la seguridad de forma agregada (si entras al dominio, heredas sus reglas).Las interacciones entre dominios están controladas (ej. la analogía del castillo: muros, foso, torres de vigilancia, puntos de entrada limitados).

Políticas (Policies) - "El Qué":
Reglas de alto nivel. Pueden superponerse y deben ser coherentes.

Principio de Mínimo Privilegio: Cada sujeto tiene solo los privilegios necesarios para su tarea.

Modelos de Permisos:
Forbid list (Lista negra): Todo permitido excepto lo prohibido.
Permit list (Lista blanca): Todo prohibido excepto lo autorizado (más seguro).
Mecanismos - "El Cómo":
Implementan las políticas.

Ejemplos técnicos: Sandboxing (confinamiento), autenticación, filtrado, logging, algoritmos criptográficos.

Controles:
Cualquier aspecto que minimice el riesgo. Incluye estándares (ISO 27001) y procesos.
Matriz de Tipos de Control (¡Importante!): Se clasifican por su naturaleza (Físico, Técnico, Administrativo) y su función (Prevención, Detección, Corrección).

Ejemplo Técnico-Preventivo: Firewall, Cifrado.

Ejemplo Físico-Detectivo: CCTV (Cámaras), Alarmas.

Ejemplo Administrativo-Correctivo: Plan de continuidad de negocio.
5. Modelos de Defensa (Evolución)
Defensa Perimetral (Perimeter Defense):

Protege contra atacantes externos (Internet).
Asume que los usuarios internos son de confianza.

Limitación: No protege contra ataques internos o credenciales robadas. Demasiado simple para empresas grandes.

Defensa en Profundidad (Defense in Depth):

Múltiples capas (cebolla). Si falla el firewall, está el antivirus; si falla este, están los permisos de usuario.

Protege contra internos y externos.

Limitación: Costoso y complejo de coordinar. Puede haber solapamientos o agujeros si no se gestiona bien.

Zero Trust (Confianza Cero):

Modelo moderno sin perímetros específicos.

Principio clave: No hay confianza inherente. Estar "dentro" de la red no te da privilegios automáticos.

Es lo recomendado para sistemas nuevos. Los sistemas antiguos (legacy) necesitan adaptadores para funcionar así.

6. Seguridad Práctica

Imposibilidad de perfección: La seguridad perfecta no existe. Enfócate en los eventos más probables.

Coste vs. Impacto: No gastes más en el collar que en el perro, pero recuerda que no hay límite en el costo de una estrategia de seguridad (el daño a la marca puede ser infinito).

Gestión del Atacante: Define controles específicos para tus atacantes probables, pero asume que siempre habrá alguien con más recursos. Ten planes de recuperación listos.

PDF 2: VULNERABILIDADES

1. Relación entre Amenazas, Ataques y Vulnerabilidades
Debes entender el flujo lógico de un incidente de seguridad, ya que es la base del modelo defensivo:
Amenazas (Threats): Generan ataques.
Ataques (Attacks): Exploran vulnerabilidades.

Vulnerabilidades (Vulnerabilities): Son fallos que, al ser explotados, incrementan el impacto sobre el activo (valor).

Defensas:

La Prevención reduce las vulnerabilidades.

La Recuperación reduce el impacto.

La Detección descubre ataques.

2. Gestión del Riesgo (La Fórmula)
En ingeniería, el riesgo no es una "sensación", es un cálculo.

formula :
Riesgo = Impacto x Probabilidad

Interpretación:
Alto impacto y alta probabilidad = Riesgo muy alto.
Para reducir el riesgo, debes disminuir el impacto, la probabilidad, o ambos.

Tipos de Impacto:
Material (Tangible): Pérdidas financieras, multas legales (GDPR), paradas de producción, costes de recuperación.

Inmaterial (Intangible): Daño a la reputación, pérdida de confianza del cliente, estrés en los empleados (burnout), tensiones geopolíticas.

3. Clasificación de Medidas de Seguridad
El documento categoriza las herramientas según su función. Esto es muy útil para preguntas de tipo "relaciona la herramienta con su objetivo":

Desaliento (Discouragement): Castigos legales, barreras de seguridad.

Prevención: Firewalls, Autenticación, Sandboxing, Parcheo de vulnerabilidades (updates).
Detección: IDS (Sistemas de Detección de Intrusos como Snort), Auditorías.
Engaño (Deception): Honeypots (sistemas trampa para atraer atacantes).
Recuperación: Backups, sistemas redundantes.

4. Estándares de Vulnerabilidades (¡Vital para el examen!)
Debes saber diferenciar estas tres siglas sin dudar:

A. CVE (Common Vulnerabilities and Exposures)
Qué es: Un diccionario de vulnerabilidades de seguridad pública conocidas.

Objetivo: Proporcionar identificadores comunes y únicos para que diferentes herramientas de seguridad puedan intercambiar datos.

Formato: CVE-Año-Orden (ej. CVE-2023-1234).

B. CVSS (Common Vulnerability Scoring System)

Qué es: El sistema para puntuar la severidad (urgencia e impacto).
Escala (0 a 10):
0.0 – 3.9: Baja.
4.0 – 6.9: Media.
7.0 – 8.9: Alta.
9.0 – 10.0: Crítica (Riesgo muy alto, requiere parche urgente).

C. CWE (Common Weakness Enumeration)

Qué es: Una lista de los tipos de errores o causas raíz (software o diseño) que generan las vulnerabilidades.

Diferencia: Mientras CVE es la instancia específica (el "bicho"), CWE es la categoría del fallo (ej. "Desbordamiento de búfer").

Relación: Se usa para categorizar los fallos, como en el OWASP Top 10 (lista de las 10 vulnerabilidades web más críticas).

5. Conceptos Técnicos: Vulnerabilidad vs. Exposición
A veces se usan como sinónimos, pero técnicamente no lo son:

Vulnerabilidad: Un error que permite a un atacante violar directamente una política de seguridad (ej. ejecutar comandos como otro usuario, DoS, hacerse pasar por otro).

Exposición (Exposure): Un error o configuración que actúa como trampolín. No permite el compromiso directo por sí solo, pero facilita la recolección de información o es un componente de un ataque mayor.

6. Zero Day y Equipos de Respuesta

Ataque Zero Day (Día Cero): Ataque que usa una vulnerabilidad desconocida para el fabricante y para la cual no existe parche. Puede existir durante meses antes de ser descubierta.

CERT (Computer Emergency Response Team): Nació tras el "Gusano Morris" (1988). Se enfoca en coordinar respuestas a emergencias y divulgar vulnerabilidades.

CSIRT (Computer Security Incident Response Team): Tiene un enfoque más amplio que solo emergencias: monitoreo regular, análisis forense, mejora continua y gestión completa de incidentes.

PDF 3.1 : FUNCIONES DIGEST

1. Definición y Propiedades (Lo más teórico)

Una función de resumen (*Digest function* o *Hash*) toma un texto de longitud variable y produce un valor de longitud fija. Funciona como una "huella digital" del archivo.

Propiedades Obligatorias (Pregunta de examen):

Para que sea criptográficamente segura, debe cumplir 3 resistencias:

1. **Resistencia a la Preimagen (Unidireccionalidad):** Dado un hash, es imposible recuperar el texto original.
2. **Resistencia a la 2.ª Preimagen:** Dado un texto A, es imposible encontrar otro texto B que genere el mismo hash.
3. **Resistencia a Colisiones:** Es imposible encontrar *cualesquiera* dos textos que den el mismo hash.
    - *Nota:* Aquí entra la **Paradoja del Cumpleaños (Birthday Paradox)**, que explica por qué es más fácil encontrar una colisión al azar que romper la preimagen.

2. Arquitecturas Internas

¿Cómo se construyen estas funciones? El PDF menciona dos enfoques:

- **Merkle-Damgård:** Compresión iterativa por bloques con relleno (padding). Es la estructura clásica.
- **Funciones Esponja (Sponge):** Tienen dos fases, *Absorción* (el sistema "chupa" los datos a un estado interno) y *Exprimido* (saca el hash del estado interno).

3. El "Zoológico" de Algoritmos

Debes saber cuáles están "muertos" y cuáles son el estándar actual:

- **Inseguros / Rotos:**
    - **MD5:** Muy fácil encontrar colisiones. Solo usar si la seguridad no importa (ej. verificar que un archivo bajó bien, pero no para contraseñas).
    - **SHA-1:** Ya no es seguro (colisiones encontradas en 2017).
- **Seguros / Estándares:**
    - **SHA-2:** (SHA-256, SHA-512). El estándar actual más común.
    - **SHA-3:** La nueva generación.

4. Aplicaciones Críticas: MIC vs MAC

Esta es la distinción más importante para un ingeniero.

A. MIC (Message Integrity Code) - Sin Clave

- **Objetivo:** Detectar cambios accidentales (errores de red, disco duro).
- **Funcionamiento:** Envías el Texto + Hash(Texto).
- **Debilidad:** No protege contra ataques inteligentes. Si un hacker cambia el texto, puede recalcular el hash nuevo y enviarlo. El receptor no notará la diferencia.

B. MAC (Message Authentication Code) - Con Clave

- **Objetivo:** Autenticar el mensaje. Asegura integridad Y origen.
- **Funcionamiento:** El hash se calcula usando una **clave secreta** compartida (K).
- **Seguridad:** Solo quien tenga la clave K puede generar o validar el hash válido. Un atacante puede cambiar el texto, pero no puede generar el MAC correcto sin la clave.

5. HMAC (Hash-based MAC)

Es una forma estandarizada y segura de construir un MAC usando funciones hash.

- No basta con pegar la clave al texto (Key + Text), eso es vulnerable.
- **Estructura anidada:** Usa dos pasadas de hash con rellenos especiales llamados ipad (inner padding) y opad (outer padding).

$$HMAC(K, m) = H((K \oplus opad) \mid H((K \oplus ipad) \mid m))$$

PDF 3.2 : KEY DERIVATION

Aquí tienes el resumen estructurado para tu examen de SIC, basado en `3.2-Key_derivation-en.pdf`:

1. El Problema (Motivación)

Los algoritmos de cifrado (como AES) necesitan claves de **longitud fija** (128, 256 bits) y **alta entropía** (aleatoriedad).

- **El conflicto:** Los humanos usamos contraseñas de longitud variable, cortas y predecibles (baja entropía).
- **La solución:** Usar una **KDF (Key Derivation Function)** para transformar esa contraseña débil en una clave técnica robusta.

2. Objetivos de la Derivación

No solo es transformar, se busca cumplir dos metas:

1. **Refuerzo de Clave (Key Reinforcement):** Hacer que los ataques de diccionario o fuerza bruta sean imprácticos. Se logra haciendo el proceso **lento** a propósito.
2. **Expansión de Clave (Key Expansion):** Generar una clave del tamaño exacto necesario o derivar múltiples claves (ej. una para cifrar y otra para autenticar/MAC) desde un solo secreto maestro.

3. Ingredientes de una KDF

Para derivar una clave necesitas cuatro elementos básicos:

1. **Secreto inicial:** La contraseña o semilla.
2. **Longitud final:** Cuántos bits necesitas.
3. **Salt (Sal):** Un valor aleatorio público que hace la derivación única. **Vital para el examen:** El *salt* evita que dos usuarios con la misma contraseña tengan la misma clave final y protege contra *Rainbow Tables*.
4. **Coste (Dificultad):** Un parámetro ajustable para hacer la función lenta.

4. Evolución de los Algoritmos (De simple a complejo)

A. Derivación Elemental (Insegura hoy en día)

- **Fórmula:** `Clave = Hash(Secreto, Salt)`.
- **Problema:** Aunque usas *Salt*, el cálculo es demasiado rápido. Un atacante con una GPU moderna puede probar millones de contraseñas por segundo.

B. PBKDF2 (El estándar clásico)

- **Significado:** *Password Based Key Derivation Function 2*.
- **Mecanismo:** Aplica una función pseudo-aleatoria (como HMAC) miles de veces (iteraciones o *rounds*).
- **Defensa:** Al obligar a hacer 100,000 iteraciones, ralentizas al usuario legítimo solo un poco (0.1 seg), pero ralentizas al atacante inmensamente (le tomaría años probar todo el diccionario).
- **Limitación:** Es vulnerable a ataques con hardware dedicado (ASICs o GPUs) porque requiere poca memoria.

C. Algoritmos "Memory-Hard" (Lo moderno)

Para combatir a las GPUs y ASICs (que tienen mucha potencia de cálculo pero poca memoria por núcleo), nacieron algoritmos que exigen **mucha RAM** para funcionar:

1. **scrypt:** Basado en PBKDF2 pero añade coste de memoria. Si no tienes la RAM, no puedes calcularlo eficientemente.
2. **Argon2:** El ganador actual de las competiciones de hashing. Permite ajustar tanto el coste de **tiempo** (CPU) como el de **memoria** (RAM) y paralelismo. Es el recomendado hoy en día.

PDF 3.3 : SIMETRIA MODERNA

1. Conceptos Fundamentales

- **Cifrado Simétrico:** Se usa la **misma clave** para cifrar y descifrar.
    - *Uso:* Protección personal (archivos en disco) o comunicación segura (si ambos comparten la clave previamente).
- **Los dos principios de diseño (Shannon):**
    - **Confusión:** La relación entre la clave y el texto cifrado debe ser compleja.
    - **Difusión:** Si cambias **un solo bit** del texto original, deben cambiar **muchos bits** en el resultado (Efecto Avalancha).

2. Tipos de Cifradores: Flujo vs. Bloque (¡Vital!)

A. Cifradores de Flujo (Stream Ciphers)

- **Cómo funcionan:** Cifran bit a bit (o byte a byte) mezclando el texto plano con un "flujo de claves" (*keystream*) usando la operación **XOR** (oplus).
- **Generador:** Usan una semilla (Clave + Vector de Inicialización IV) para crear un flujo pseudoaleatorio infinito.
- **Regla de Oro (Pregunta de examen): NUNCA** reutilices la misma Clave y IV para dos mensajes distintos. Si lo haces, un atacante puede anular la clave y leer el mensaje haciendo un simple XOR entre los dos criptogramas.
- **Integridad:** Son muy vulnerables a manipulaciones (bit-flipping) si no tienen controles de integridad adicionales.
- *Ejemplos:* RC4 (roto, usado en WEP antiguo), ChaCha20, A5/1 (GSM).

B. Cifradores de Bloque (Block Ciphers)

- **Cómo funcionan:** Dividen el texto en bloques de tamaño fijo (ej. 64 o 128 bits) y cifran cada bloque por separado.
- **Estructuras internas:** Usan rondas repetitivas de sustitución y permutación.
    - *Redes Feistel* (usado en DES).
    - *Redes de Sustitución-Permutación (SPN)* (usado en AES).

3. Los Algoritmos "Famosos"

- **DES (Data Encryption Standard):**
    - Muy viejo (1977). Bloques de 64 bits.
    - **Debilidad:** Clave muy corta (56 bits), hoy en día se rompe por fuerza bruta fácilmente.
- **3DES (Triple DES):**
    - Parche para alargar la vida de DES. Aplica DES tres veces ($Cifrar \to Descifrar \to Cifrar$). Es seguro pero muy lento (3 veces más lento).
- **AES (Advanced Encryption Standard) - El Rey Actual:**
    - Estándar desde 2001. Bloques de 128 bits.
    - Claves de 128, 192 o 256 bits.
    - **Funcionamiento interno:** 4 fases por ronda (SubBytes, ShiftRows, MixColumns, AddRoundKey). *No necesitas memorizar la matemática exacta, pero sí los nombres de las fases*.

4. Modos de Operación (¡Ojo aquí!)

Un algoritmo de bloque (como AES) solo cifra un bloque de 16 bytes. ¿Cómo cifras un archivo de 1GB? Usando un **Modo de Operación**.

1. **ECB (Electronic Code Book) - El "Malo":**
    - Cifra cada bloque independientemente.
    - **Problema:** Bloques de texto idénticos generan bloques cifrados idénticos. **No oculta patrones** (recuerda la imagen del Pingüino de Linux que se ve igual cifrado)^{11111111}.
    - *Ventaja:* Paralelizable.
2. **CBC (Cipher Block Chaining) - El Clásico:**
    - Encadena los bloques. El resultado del bloque anterior se mezcla (XOR) con el actual antes de cifrar.
    - Requiere un **IV (Vector de Inicialización)** aleatorio para el primer bloque.
    - **Problema:** El cifrado es **secuencial** (no se puede paralelizar), pero el descifrado sí es paralelo.
3. **CTR (Counter Mode) - El Moderno:**
    - Convierte un cifrador de bloque en uno de **flujo**.
    - Cifra un "contador" (1, 2, 3...) y hace XOR con el texto.
    - **Ventaja:** Totalmente paralelizable (muy rápido), acceso aleatorio (puedes descifrar el final del archivo sin leer el principio) y no necesita relleno (*padding*).
4. **GCM (Galois/Counter Mode) - El "Todo en Uno":**
    - Usa modo CTR para cifrar + una función matemática (Galois) para **autenticar**.
    - Te da **Confidencialidad + Integridad** al mismo tiempo. Es el estándar de oro hoy en día en comunicaciones web (HTTPS).

PDF 3.4 : CRIPTOGRAFIA ASIMETRICA

1. El Concepto Clave: Par de Claves

A diferencia del cifrado simétrico (una sola clave), aquí usamos un **par de claves**:

- **Clave Privada (K_{prv}):** Personal, secreta y **nunca** se comparte.
- **Clave Pública (K_{pub}):** Disponible para todo el mundo.

Lo que se cifra con una, solo se puede descifrar con la otra. Esto permite dos funciones principales que suelen ser **pregunta de examen**:

A. Confidencialidad (Quiero enviarte un secreto)

- **Mecanismo:** Alice cifra el mensaje usando la **Clave Pública de Bob**.
- **Por qué:** Solo Bob tiene la clave privada correspondiente para descifrarlo. Ni siquiera Alice puede descifrar lo que acaba de escribir.

B. Autenticación / Firma Digital (Quiero demostrar que soy yo)

- **Mecanismo:** Alice cifra (firma) el mensaje (o su hash) usando su propia **Clave Privada**.
- **Por qué:** Cualquiera puede descifrarlo con la clave pública de Alice. Si funciona, **matemáticamente** solo pudo haberlo creado Alice. Garantiza integridad y origen (no repudio).

2. Problema de Rendimiento y Solución: Cifrado Híbrido

- **El Problema:** La criptografía asimétrica es muy lenta y consume muchos recursos.
- **La Solución (Cifrado Híbrido):** Combina lo mejor de ambos mundos.
    1. Generas una clave simétrica aleatoria (rápida) temporal.
    2. Cifras los datos grandes con esa clave simétrica.
    3. Cifras la clave simétrica "pequeña" usando la clave pública del receptor (asimétrico).
    4. Envías todo junto ($C1$ datos + $C2$ clave cifrada).

3. Algoritmos Principales

A. RSA (Rivest, Shamir, Adleman)

- Basado en la dificultad de factorizar números enteros muy grandes.
- Se usa tanto para cifrar como para firmar.

B. Diffie-Hellman (DH) - Intercambio de Claves

- **Objetivo:** Permitir que dos personas acuerden una clave secreta a través de un canal inseguro sin haberse visto antes.
- **Funcionamiento:** Se basa en logaritmos discretos. Alice y Bob mezclan sus claves privadas con valores públicos para llegar al mismo número secreto común (K_{ab}).
- **Vulnerabilidad (¡Importante!):** Es vulnerable al ataque **Man-in-the-Middle (MitM)** Como DH no autentica a las partes, un atacante puede ponerse en medio, acordar una clave con Alice y otra con Bob, y leer todo el tráfico.

C. ECC (Elliptic Curve Cryptography)

- Usa curvas matemáticas en lugar de solo números primos grandes.
- **Ventaja:** Ofrece la misma seguridad que RSA pero con **claves mucho más pequeñas** y eficientes. Ideal para móviles o tarjetas inteligentes.
- **ECDH:** Es la versión de Diffie-Hellman usando curvas elípticas.

4. Aleatorización (OAEP)

Si cifras el mismo mensaje (ej. "Hola") con la misma clave pública RSA varias veces, el resultado es el mismo. Esto permite ataques de "prueba y error".

- **Solución:** Usar esquemas de relleno (*padding*) como **OAEP**. Añade aleatoriedad (ruido) antes de cifrar, asegurando que el mismo texto plano siempre produzca criptogramas diferentes.

PDF 3.5 : MAC FUNCTIONS

1. Definición y Propósito

Un **MAC (Message Authentication Code)** es un mecanismo criptográfico utilizado para autenticar mensajes. Se puede definir como un Código de Integridad de Mensaje (MIC) calculado utilizando una **clave secreta**.

- **Función Principal:** Garantiza tanto la **integridad** de los datos (que no han sido modificados) como la **autenticidad** del origen (que quien lo envió posee la clave).
- **Validación:** Solo los poseedores de la clave pueden generar o validar el MAC.

2. Enfoques para Construir un MAC

Existen varias formas de implementar funciones MAC:

- **Cifrado de un resumen (Digest):** Usando un cifrador de bloque simétrico sobre el hash del mensaje.
- **Cifrado con retroalimentación (CBC-MAC):** Utiliza cifrado con propagación de errores para generar el código.
- **HMAC (Hash-based MAC):** Es el enfoque más común y añade una clave a los datos en los que se aplica el hash.
    - Estructura del HMAC: No es una simple concatenación. Utiliza una estructura anidada para mayor seguridad:
      H(K oplus opad, H(K oplus ipad, texto))
    - Donde ipad y opad son constantes de relleno específicas.

3. Combinación de Cifrado y Autenticación

Cuando se requiere confidencialidad (cifrado) e integridad (MAC) simultáneamente, el orden de las operaciones es crítico para la seguridad:

1. **Encrypt-then-MAC (Cifrar luego MAC):**
    - Se calcula el MAC sobre el *criptograma* (texto cifrado).
    - **Ventaja:** Es la opción preferible porque permite verificar la integridad antes de descifrar, protegiendo contra ataques al proceso de descifrado.
2. **Encrypt-and-MAC:** Se calcula el MAC sobre el texto plano y se envía junto al cifrado. El MAC no está cifrado.
3. **MAC-then-Encrypt:** Se calcula el MAC sobre el texto plano y luego se cifra todo junto. Requiere descifrar todo antes de validar.

4. Modo GCM (Galois Counter Mode)

El documento destaca el GCM como un modo moderno y eficiente de cifrado autenticado.

- **Funcionamiento:**
    - Utiliza el **Modo Contador (CTR)** para el cifrado.
    - Utiliza la **Multiplicación de Galois** para generar la etiqueta de autenticación (*auth tag*).
- **Características:** Permite procesar datos de autenticación adicionales (AAD) que no están cifrados pero sí autenticados.

1. Definición y Propósito

La **firma digital** es el equivalente electrónico de una firma manuscrita, pero con garantías matemáticas mucho más fuertes. Se basa en la criptografía asimétrica para proporcionar:

- **Autenticación de Contenido (Integridad):** Garantiza que el documento no ha sido modificado ni un solo bit desde que fue firmado.
- **Autenticación de Origen:** Asegura la identidad de quien creó la firma.
- **No Repudio:** El autor genuino no puede negar haber firmado el documento, ya que matemáticamente solo él (con su clave privada) pudo haber generado esa firma.

2. Mecanismo de Funcionamiento

El proceso se basa en el uso de un par de claves (pública y privada) y funciones hash:

A. Proceso de Firma (El Emisor):

1. Se calcula el **hash** (resumen) del documento o datos ($Digest(doc)$).
    - *Nota:* Se firma el hash y no el documento entero por razones de **rendimiento** (es mucho más rápido).
2. Se **cifra** ese hash usando la **Clave Privada** del firmante.
3. El resultado es la Firma Digital, que se adjunta al documento original.

B. Proceso de Verificación (Cualquier Receptor):

1. Se toma el documento recibido y se calcula su hash de nuevo.
2. Se toma la firma digital y se **descifra** usando la **Clave Pública** del firmante.
3. **Comparación:**
    - Si $Hash_{calculado} == Hash_{descifrado}$, la firma es **VÁLIDA**.
    - Si son diferentes, el documento fue alterado o la firma no corresponde a esa clave pública.

3. Algoritmos Comunes

El documento menciona los algoritmos estándar para realizar firmas digitales:

- **RSA:** Utiliza la operación de exponenciación modular. Se usa la clave privada para "cifrar" (firmar) y la pública para "descifrar" (verificar).
- **DSA / ElGamal:** Algoritmos diseñados específicamente para firma digital.
- **ECDSA (Curvas Elípticas):** Versión eficiente basada en curvas elípticas, muy usada hoy en día.

4. Estructura de un Documento Firmado (Ejemplo Real)

El PDF muestra cómo se ve un correo electrónico firmado digitalmente (S/MIME):

- Es un contenido **Multipart/signed**.
- Contiene el mensaje original en texto plano.
- Contiene un adjunto (ej. smime.p7s) que es la firma criptográfica en sí misma, codificada en Base64.

1. Objetivos de la Gestión de Claves

La seguridad de un sistema asimétrico no depende solo del algoritmo, sino de la gestión correcta de las claves a lo largo de su ciclo de vida:

- **Generación:** Cuándo y cómo se crean los pares de claves.
- **Manejo:** Cómo se protegen las claves privadas para evitar robos.
- **Distribución:** Cómo hacer llegar las claves públicas a todo el mundo de forma fiable.
- **Vida útil:** Definir cuándo caducan y cómo gestionar la obsolescencia.

2. Principios de Seguridad

El documento establece reglas claras para mantener la confianza:

- **Auto-generación:** Las claves privadas deben ser generadas por el propio dueño (self-generation). Esto maximiza la privacidad, ya que nadie más (ni siquiera la CA) tiene copia.
- **Protección de la Clave Privada:**
    - Debe estar protegida por contraseña o PIN.
    - Se pueden tener copias de seguridad (backups) físicas seguras.
    - Es vital asegurar que las aplicaciones que usan la clave no tengan fugas de información.

3. Revocación de Certificados

Cuando una clave privada se compromete o los datos cambian, el certificado debe ser revocado antes de que caduque.

- **Mecanismos:**
    - **CRL (Certificate Revocation List):** Listas publicadas periódicamente por la CA con los certificados anulados.
    - **OCSP (Online Certificate Status Protocol):** Servicio para consultar en tiempo real el estado de un certificado.

4. Mecanismos de Defensa Avanzados

El PDF destaca dos técnicas modernas para mitigar las debilidades del sistema de CAs tradicional:

A. Certificate Pinning (Anclaje de Certificados)

- **Problema:** Si una CA raíz es comprometida, puede emitir certificados falsos para cualquier sitio.
- **Solución:** Incluir ("anclar") la huella digital (*fingerprint* o hash) de la clave pública legítima directamente en el código fuente de la aplicación.
- **Validación:** La aplicación solo acepta el certificado si su huella coincide con la que tiene guardada, ignorando la cadena de confianza estándar si es necesario.

B. Certification Transparency (RFC 6962)

- **Problema:** Los compromisos de CAs (como el caso DigiNotar) son difíciles de detectar por los usuarios o dueños de dominios.
- **Solución:** Un sistema global de registros (*logs*) públicos donde se anotan todos los certificados emitidos.
- **Objetivo:** Permitir auditorías para asegurar que ningún certificado ha sido emitido en secreto o fraudulentamente.

PDF 5 : AUTENTICACION

1. Conceptos Fundamentales

La **Autenticación (Authn)** es el proceso de probar una identidad reclamada ("Soy Joe" $\to$ "Pruébalo"). Se distingue claramente de la **Autorización (Authz)**, que determina qué puede hacer esa identidad una vez verificada.

Factores de Autenticación (Tipos de Pruebas):

- **Algo que sabes:** Contraseñas, PINs.
- **Algo que tienes:** Tokens físicos, tarjetas inteligentes, el teléfono móvil.
- **Algo que eres:** Biometría (huella, iris).
- **MFA (Multi-Factor Authentication):** Combinar dos o más de los anteriores para mayor seguridad.

2. Desafíos y Soluciones Básicas

El envío de contraseñas en texto claro es peligroso (sniffing). Para evitarlo se usan mecanismos más robustos:

- **Desafío-Respuesta (Challenge-Response):** El servidor envía un número aleatorio (*nonce*) y el usuario debe responder con un cálculo que involucre su contraseña y ese número. Esto evita ataques de repetición (*Replay Attacks*).
- **Contraseñas de un solo uso (OTP):** Como el esquema de Lamport, donde se usa una cadena de hashes inversa para generar contraseñas que solo valen una vez.

3. Protocolos de Red (AAA)

- **RADIUS:**
    - Estándar para acceso a redes (ISPs, WiFi empresarial como Eduroam, VPNs).
    - Modelo Cliente-Servidor.
    - Proporciona **A**utenticación, **A**utorización y **A**ccounting (Auditoría/Cobro).

4. Kerberos (El estándar de SSO en LAN)

Es el protocolo por defecto en entornos Windows/Active Directory.

- **Objetivo:** Permitir **Single Sign-On (SSO)**. Te autenticas una vez y accedes a todos los recursos (impresoras, carpetas) sin volver a poner la clave.

- **Funcionamiento:**
    - Usa un tercero de confianza llamado **KDC** (Key Distribution Center).
    - Funciona a base de **Tickets**. Nunca envía la contraseña por la red.
    - Requiere sincronización de relojes (timestamp) para evitar ataques de replay.

5. Autenticación Web y Federada (Moderno)

Cuando usas "Entrar con Google" o "Entrar con Facebook", estás usando estos protocolos:

A. SAML (Security Assertion Markup Language)

- Basado en **XML**.
- Muy usado en entornos empresariales antiguos y federaciones de identidad. Es potente pero pesado.

B. OAuth 2.0 (¡Ojo! Es para Autorización)

- Diseñado para **delegación de permisos**. (Ej: "Permitir a esta App imprimir mis fotos de Google Drive").
- No fue diseñado para autenticar, aunque se usó mal para eso muchas veces.
- Usa **Access Tokens** para acceder a recursos.

C. OpenID Connect (OIDC)

- Es una capa de **identidad** construida *encima* de OAuth 2.0.
- Añade un **ID Token** (formato JSON/JWT) que dice "quién es el usuario".
- Es el estándar moderno para autenticación en aplicaciones móviles y web.

PDF 6.2 : ZIGBEE

1. Contexto y Definición

ZigBee es un estándar de comunicación inalámbrica de bajo consumo y baja tasa de transferencia (20-250 kbps), ideal para redes de sensores e **Internet de las Cosas (IoT)**.

- **Base Tecnológica:** Funciona sobre el estándar **IEEE 802.15.4**.
- **Alcance:** Corto alcance (10-100 metros).
- **Seguridad Base:** Implementa cifrado **AES-128** y control de integridad a nivel de hardware.

2. Tipos de Dispositivos y Topología

La seguridad depende de los roles que juegan los dispositivos en la red:

- **FFD (Full-Function Device):** Puede actuar como Coordinador o Router. Habla con cualquier nodo y puede retransmitir mensajes.
- **RFD (Reduced-Function Device):** Dispositivos finales (sensores simples). Solo hablan con su coordinador, no retransmiten.
- **Topologías:** Soportan Estrella (Star), Árbol (Cluster Tree) y Malla (Mesh/Peer-to-Peer).

3. Jerarquía de Claves (Key Hierarchy)

Este es el punto crítico para el examen. ZigBee maneja tres tipos de claves para proteger la comunicación:

1. **Network Key (Clave de Red):**
    - Es una clave de 128 bits **compartida por todos** los dispositivos de la red (PAN).
    - Se usa para cifrar tráfico de **difusión (broadcast)** y tráfico de control.
    - **Rotación:** Posee un número de secuencia (0-255). Cuando se actualiza la clave, se incrementa el número para que los dispositivos sepan cuál usar.
2. **Link Key (Clave de Enlace):**
    - Clave única compartida solo entre **dos nodos específicos**.
    - Se usa para proteger la comunicación **unicast** (punto a punto).
    - Se deriva a partir de la Master Key.
3. **Master Key (Clave Maestra):**
    - Secreto pre-compartido (instalado en fábrica o manualmente).
    - Su única función es servir de base para derivar las *Link Keys* mediante el protocolo **SKKE** (Symmetric-Key Key Establishment).

4. El Centro de Confianza (Trust Center)

Es el "policía" de la red ZigBee (generalmente el Coordinador PAN).

- **Funciones:**
    - Decide si admite o no a un nuevo dispositivo en la red.
    - Distribuye la **Network Key** a los dispositivos nuevos tras autenticarlos.
    - Gestiona la renovación de claves.
- **Proceso de Unión:** Un dispositivo nuevo se une temporalmente hasta que el Trust Center lo autentica y le envía la clave de red. Si falla, es expulsado.

PDF 6.3 : LORAWAN

1. Conceptos Básicos y Arquitectura

Es fundamental distinguir entre **LoRa** (la capa física de radio, largo alcance y bajo consumo) y **LoRaWAN** (el protocolo de red que corre encima).

- **Objetivo:** Conectar dispositivos (sensores) que envían pocos datos a lo largo de varios kilómetros con muy poca batería.
- **Arquitectura de Estrella de Estrellas (Star-of-Stars):**
    - **End Device:** El sensor.
    - **Gateway:** Antenas que reciben la señal de radio y la pasan a IP. No descifran nada, son simples "buzones" transparentes.
    - **Network Server:** El cerebro de la red. Gestiona la autenticación, deduplicación de paquetes y control de red.

    - **Application Server:** Donde llegan los datos finales descifrados.

2. Métodos de Activación (Cómo unirse a la red)

Para que un dispositivo hable seguro, debe "registrarse". Hay dos formas y suelen ser **pregunta de examen**:

A. OTAA (Over-The-Air Activation) - La Opción Segura

- **Dinámico:** El dispositivo y la red negocian claves de sesión nuevas cada vez que se unen.
- **Requisitos:** El dispositivo tiene grabadas de fábrica dos claves raíz (`AppKey`, `NwkKey`) y un identificador (`DevEUI`).
- **Proceso:** Envía un *Join-request* y, si es válido, el servidor responde con un *Join-accept*. De ese intercambio se derivan matemáticamente las claves de sesión.
- **Ventaja:** Si las claves de sesión se comprometen, basta con reiniciar el dispositivo para generar otras nuevas.

B. ABP (Activation By Personalization) - La Opción Simple/Insegura

- **Estático:** No hay negociación ("Join"). Las claves de sesión (`AppSKey`, `NwkSKey`) se graban a fuego (hardcoded) en el dispositivo.
- **Uso:** Dispositivos muy simples o redes privadas específicas.
- **Riesgo:** Si se roban las claves, no se pueden cambiar remotamente; hay que reprogramar el chip físico.

3. Esquema de Seguridad (Las Claves)

LoRaWAN usa **AES-128** y separa la seguridad de la red de la seguridad de los datos (payload). Esto significa que el operador de la red (Telecom) no puede leer tus datos (Application).

1. **NwkSKey (Network Session Key):**
    - Compartida entre el **Dispositivo** y el **Network Server**.
    - **Función:** Integridad del paquete (firma MIC) y cifrado de comandos MAC (control de red).
2. **AppSKey (Application Session Key):**
    - Compartida entre el **Dispositivo** y el **Application Server**.
    - **Función:** Cifrado del payload (los datos del sensor). El Network Server ve estos datos cifrados y no puede leerlos.

4. Protección contra Ataques

- **Anti-Replay (Ataques de repetición):** Se usan contadores de tramas (*Frame Counters*). Si el servidor recibe un paquete con un contador menor o igual al último recibido, lo descarta silenciosamente.
- **Integridad:** Se usa AES-CMAC para calcular el código de integridad (MIC) de cada trama. Si se modifica un bit en el aire, el MIC no cuadra y se descarta.

PDF 7.1 : VPN

1. Definición y Objetivo

Una **VPN (Virtual Private Network)** no es solo una herramienta para cambiar tu IP, en ingeniería se define como la **extensión de un perímetro de seguridad** a través de una red insegura (como Internet) sin reducir la seguridad global.

- **Usos Clave:**
    - Acceso remoto a redes organizacionales (ej. trabajar desde casa conectándose a la red de la universidad).
    - Interconexión segura de dos redes separadas geográficamente.
- **Requisitos de Gestión:** Para que funcione, requiere autenticación robusta de usuarios, comunicaciones seguras (cifrado) y control de acceso a los servicios internos.

2. Mecanismos de Funcionamiento

El PDF destaca dos mecanismos principales para crear una VPN:

A. Tunelización Segura (Secure Tunneling)

- Permite **encapsular** protocolos dentro de otros protocolos.
- Todo lo que viaja dentro del túnel está protegido.
- Permite realizar control de tráfico en la entrada/salida del túnel.

B. Mediación (Proxying)

- Uso de intermediarios (routers, gateways) para explotar enlaces VPN de forma transparente.
- Los **Firewalls** actúan como la frontera natural del perímetro de seguridad y suelen ser el punto donde termina la VPN.

3. Implementación Técnica (Cómo se construyen)

El documento explica cómo se implementan las VPNs a nivel de sistema operativo (Linux), lo cual es muy técnico y "preguntable":

- **Dispositivos TUN/TAP (Linux):**
    - **TUN:** Simula un dispositivo de **Capa 3 (Red/IP)**. Se usa para enrutamiento.
    - **TAP:** Simula un dispositivo de **Capa 2 (Enlace/Ethernet)**. Se usa para puentes (bridges).
    - **OpenVPN:** Es una aplicación en *User-Space* (espacio de usuario) que interactúa con estas interfaces virtuales para redirigir el tráfico hacia la red real cifrada.
- **WireGuard:**

    - Tecnología más moderna. Implementa las interfaces de túnel directamente dentro del **Kernel** (núcleo) para mayor rendimiento.
    - Los extremos se identifican con **Claves Públicas** (no solo IPs), lo que permite que cambien de dirección IP (roaming) sin cortar la conexión.

4. Cifrado Oportunista (Opportunistic Encryption)

Es un enfoque de "mejor esfuerzo" (best-effort).

- **Filosofía:** Si el cliente soporta cifrado, úsalo. Si no, usa la versión insegura (fallback).
- **Ejemplos:** HTTPS vs HTTP, STARTTLS en correo electrónico.
- Se intentó hacer con IPsec, pero resultó ser un protocolo demasiado complejo para esto.

PDF 7.2 : IPSEC

1. Objetivos y Modos de Operación

IPsec es un conjunto de protocolos diseñados para proteger el tráfico a nivel de la capa IP (Capa 3). Sus objetivos principales son garantizar la **confidencialidad** y la **integridad** de los datagramas.

Ofrece dos modos de funcionamiento clave:

- **Modo Transporte:**
    - Mantiene la cabecera IP original.
    - Protege solo la carga útil (payload). Raramente usado hoy en día.
- **Modo Túnel:**
    - Encapsula el paquete IP original completo dentro de un nuevo paquete.
    - Es la base de la mayoría de las VPNs.

2. Protocolos Principales: AH y ESP

IPsec utiliza dos protocolos distintos para aplicar seguridad, que pueden usarse solos o combinados:

A. AH (Authentication Header - RFC 4302)

- **Función:** Proporciona integridad y autenticación de origen.
- **Mecanismo:** Añade un hash con clave (MAC) que cubre **todo el datagrama IP** (cabeceras + datos).
- **Limitación:** No cifra los datos (no hay confidencialidad).

B. ESP (Encapsulating Security Payload - RFC 4303)

- **Función:** Proporciona confidencialidad (cifrado) y opcionalmente integridad.
- **Mecanismo:** Cifra la carga útil y añade una cola (trailer) con datos de autenticación.
- **Detalle:** En modo túnel, cifra todo el paquete IP original.

3. Gestión de la Seguridad: SAs y Bases de Datos

Para que dos equipos hablen IPsec, deben ponerse de acuerdo en las reglas:

- **Security Association (SA):** Es un acuerdo unidireccional entre dos partes sobre qué algoritmos y claves usar.
- **SPI (Security Parameter Index):** Un número que identifica qué SA se debe usar para descifrar un paquete entrante.
- **Bases de Datos:**
    - **SPD (Security Policy Database):** Define el **"qué"** (políticas: "el tráfico a X debe ir cifrado").
    - **SAD (Security Association Database):** Define el **"cómo"** (claves y algoritmos activos).

4. Configuración y Problemas con NAT

- **Establecimiento:** Las SAs pueden configurarse manualmente o, más comúnmente, de forma automática usando el protocolo **IKE** (Internet Key Exchange).
- **El Problema del NAT:**
    - NAT modifica las cabeceras IP (cambia IPs) y de transporte (cambia puertos) para funcionar.
    - Esto rompe IPsec: **AH** falla porque el hash no coincide (la IP cambió) y **ESP** falla porque oculta los puertos necesarios para el multiplexado.
- **La Solución (NAT-Traversal):** Encapsular los paquetes IPsec dentro de paquetes **UDP**. Esto permite atravesar los routers NAT sin romper la criptografía.

PDF 7.3 : WIREGUARD

1. Conceptos Fundamentales

WireGuard se presenta como un túnel VPN rápido, moderno y seguro. Se diseñó para resolver la complejidad excesiva de los protocolos tradicionales.

- **Simplicidad Extrema:** Su código base tiene solo unos pocos miles de líneas (vs. cientos de miles en OpenVPN o IPsec), lo que lo hace mucho más fácil de auditar y mantener.
- **Rendimiento:** Funciona directamente en el **Kernel** de Linux, ofreciendo un throughput muy alto y latencia mínima.
- **Configuración Sencilla:** No usa certificados complejos X.509. Se configura con pares de claves simples (como SSH): una Clave Privada, una Pública y la lista de IPs permitidas (*AllowedIPs*).

2. Criptografía "Opinada" (Sin opciones)

A diferencia de IPsec, que permite negociar cientos de combinaciones de cifrado (lo que lleva a errores), WireGuard usa un set fijo y moderno de primitivas criptográficas:

- **Intercambio de claves:** Curve25519 (ECDH).

- **Cifrado:** ChaCha20 (muy rápido en software, incluso en móviles).
- **Autenticación (MAC):** Poly1305.
- **Hashing:** BLAKE2s (más rápido que SHA-3).

3. Mecanismo de Funcionamiento (Cryptokey Routing)

WireGuard rompe la asociación tradicional entre túnel e IP.

- **Identidad = Clave Pública:** Los peers se identifican por su clave pública, no por su IP.
- **Roaming Nativo:** Si cambias de WiFi a 4G (cambia tu IP), el túnel no se cae ni necesita renegociar. En cuanto envías un paquete autenticado desde la nueva IP, el servidor actualiza tu "Endpoint" automáticamente.
- **Silencioso:** Si no hay datos que enviar, no envía paquetes. Es "furtivo" ante escaneos de red.

4. Protocolo de Túnel

- **Encapsulamiento:** Todo el tráfico va sobre **UDP**.
- **Handshake (NoiseIK):** Utiliza el protocolo *Noise* para establecer claves de sesión de forma muy rápida (1-RTT) cada pocos minutos para garantizar **Perfect Forward Secrecy** (si te roban la clave hoy, no pueden descifrar lo de ayer).

5. Limitaciones de Uso

Aunque es excelente técnicamente, su simplicidad tiene un coste en entornos corporativos grandes:

- No tiene gestión de usuarios centralizada, LDAP ni 2FA nativo.
- Requiere infraestructura de terceros (como **Tailscale**) para gestionar las claves de miles de usuarios.

PDF 8 : SSH

1. Definición y Objetivos

SSH es un protocolo de red criptográfico diseñado para operar sobre redes inseguras. Su misión principal es **reemplazar a los protocolos antiguos e inseguros** como Telnet, Rlogin, RSH y FTP, que enviaban las contraseñas en texto claro.

- **Funciones Principales:**
    - **Login remoto seguro:** Acceder a la terminal de otro servidor.
    - **Transferencia de archivos:** SFTP y SCP (Secure Copy).
    - **Ejecución de comandos remotos:** Automatización segura.
    - **Tunneling (Port Forwarding):** Proteger el tráfico de otras aplicaciones.

2. Arquitectura del Protocolo (Capas)

SSH no es un bloque monolítico, se divide en tres capas independientes:

1. **Transport Layer Protocol (Capa de Transporte):**
    - Se encarga del cifrado (confidencialidad), control de integridad y, muy importante, la **autenticación del servidor** (para que sepas que te conectas al servidor real y no a un atacante).
2. **User Authentication Protocol (Capa de Autenticación):**
    - Se ejecuta sobre la capa de transporte.
    - Verifica la identidad del cliente (usuario) ante el servidor.
3. **Connection Protocol (Capa de Conexión):**
    - Multiplexa el túnel cifrado en varios **canales** lógicos. Esto permite tener varias sesiones de terminal y transferencias de archivos simultáneas sobre una sola conexión TCP.

3. Mecanismos de Autenticación

El servidor autentica al cliente usando varios métodos posibles:

- **Contraseña (Password):** El método clásico. Es vulnerable a ataques de fuerza bruta o adivinación.
- **Clave Pública (Public Key):** El método recomendado.
    - El usuario genera un par de claves.
    - Copia su **clave pública** en el archivo `~/.ssh/authorized_keys` del servidor.
    - El servidor envía un desafío que solo se puede responder con la **clave privada** del usuario.
    - **Ventaja:** No se envían secretos por la red y permite login automático (sin password).
- **Host-based:** Autenticación basada en la confianza entre máquinas (poco seguro hoy en día).

4. Tunneling y Port Forwarding (¡Vital para el examen!)

SSH permite encapsular otros tráficos dentro de su túnel seguro. Debes distinguir los tres tipos:

A. Local Port Forwarding (`-L`)

- **Uso:** "Quiero acceder desde mi PC a un servicio remoto que está bloqueado por firewall".
- **Sintaxis:** `ssh -L PuertoLocal:Destino:PuertoDestino usuario@servidor`
- **Flujo:** Tu PC escucha en un puerto, envía el tráfico por el túnel SSH, y el servidor SSH lo entrega al destino final.

B. Remote Port Forwarding (`-R`)

- **Uso:** "Quiero que alguien desde fuera acceda a un servicio que está en mi PC local (detrás de NAT)".
- **Sintaxis:** `ssh -R PuertoRemoto:Destino:PuertoDestino usuario@servidor`
- **Flujo:** El servidor SSH abre un puerto, y cuando alguien se conecta, el tráfico viaja hacia atrás por el túnel hasta tu PC.

C. Dynamic Port Forwarding (`-D`)

- **Uso:** Navegación anónima o saltarse restricciones geográficas.
- **Funcionamiento:** Crea un **Proxy SOCKS**. El cliente SSH actúa como un proxy dinámico y redirige el tráfico de aplicaciones (como Firefox) a través del servidor SSH hacia Internet.

5. Gestión de Claves (Archivos Importantes)

- **`known_hosts` (En el Cliente):** Guarda las claves públicas de los servidores conocidos. Protege contra ataques *Man-in-the-Middle*. Si la clave del servidor cambia, SSH te avisa con una alerta roja.
- **`authorized_keys` (En el Servidor):** Contiene la lista de claves públicas de usuarios permitidos para entrar.

PDF 9 : WIFI

1. El Problema del Medio Compartido

A diferencia de un cable, el aire es accesible para todos. Los problemas principales son:

- **Difusión (Broadcast):** Cualquiera con una antena puede escuchar (sniffing) el tráfico.
- **Límites Físicos:** No existen muros para la señal de radio, por lo que la seguridad debe implementarse en la Capa de Enlace de Datos.
- **Objetivos:** Se necesita cifrar el *payload* (carga útil) para privacidad y autenticar el origen de las tramas para evitar falsificaciones.

2. Evolución de los Protocolos de Seguridad (¡Fundamental!)

Debes conocer la historia de "fallo $\to$ parche $\to$ solución":

A. WEP (Wired Equivalent Privacy) - El "Roto"

- **Objetivo:** Dar la misma seguridad que un cable. Fracasó estrepitosamente.
- **Fallos:**
  - Usa el algoritmo **RC4** (cifrador de flujo) con un Vector de Inicialización (IV) muy pequeño (24 bits) que se repite rápido.
  - Usa **CRC-32** para integridad, que no es criptográficamente seguro (es fácil modificar el mensaje y recalcular el CRC sin conocer la clave).
  - No tiene gestión de claves dinámica (clave estática manual).

B. WPA (Wi-Fi Protected Access) - El "Parche"

- **Objetivo:** Arreglar WEP mediante software (firmware update) sin cambiar el hardware antiguo.
- **Mecanismo:** Usa **TKIP** (Temporal Key Integrity Protocol). Cambia la clave en cada paquete (Key Mixing) para que RC4 no sea tan vulnerable.
- **Integridad:** Introduce "Michael", un código de integridad de mensaje (MIC) real, aunque débil.

C. WPA2 (IEEE 802.11i) - El Estándar Robusto

- **Cambio Radical:** Requiere hardware nuevo (chip AES).
- **Cifrado:** Usa **AES** en modo **CCMP** (Counter Mode with CBC-MAC). Ofrece confidencialidad e integridad fuertes.
- **Jerarquía de Claves:**
    - **PMK (Pairwise Master Key):** La clave maestra (tu contraseña del WiFi o la del servidor RADIUS).
    - **PTK (Pairwise Transient Key):** Claves de sesión temporales derivadas de la PMK para cifrar el tráfico unicast.
    - **GTK (Group Transient Key):** Para cifrar el tráfico *broadcast* (multicast).

D. WPA3 - El Moderno

- **Mejora Crítica:** Reemplaza la autenticación PSK por **SAE (Simultaneous Authentication of Equals)**, basado en el protocolo *Dragonfly*.
- **Ventajas:**
    - Resistente a ataques de diccionario offline (si capturan tu handshake, no pueden probar contraseñas en casa).
    - **Perfect Forward Secrecy:** Si te roban la clave hoy, no pueden descifrar el tráfico capturado ayer.

3. Modos de Autenticación

- **Personal (SOHO):** Usa una **PSK (Pre-Shared Key)**. Todos los usuarios comparten la misma contraseña. Vulnerable si la contraseña es débil.
- **Enterprise (Empresarial):** Usa el estándar **IEEE 802.1X** con un servidor de autenticación (generalmente RADIUS). Cada usuario tiene sus propias credenciales (usuario/pass o certificado). Las claves de cifrado son únicas por usuario y sesión.

4. El "4-Way Handshake"

Es el protocolo que ocurre cuando te conectas al WiFi (antes de tener internet).

- **Objetivo:** Que el Cliente y el Punto de Acceso (AP) demuestren que ambos conocen la contraseña (PMK) sin enviarla por el aire, y negociar la clave de sesión (PTK).
- **Vulnerabilidad:** Es aquí donde atacan los hackers (capturan el handshake para intentar adivinar la clave PSK offline).

PDF 10 : CONTROL DE ACCESO

1. Conceptos Básicos

El control de acceso son las políticas y mecanismos que medían el acceso de un **Sujeto** (proceso, usuario) a un **Objeto** (archivo, CPU, memoria).

- **Monitor de Referencia:** Es el componente del sistema operativo que intercepta *todas* las llamadas y decide si permite o deniega el acceso.
- **Matriz de Acceso:** Es el modelo abstracto ideal. Filas = Sujetos, Columnas = Objetos. Las celdas definen los permisos (leer, escribir).

2. DAC (Discretionary Access Control) - Control Discrecional

- **Filosofía:** El **propietario (Owner)** del objeto decide quién puede acceder. Es el modelo estándar en Windows, Linux y macOS.
- **Problema:** Es vulnerable a ataques de "Caballo de Troya". Si ejecutas un programa malicioso, este hereda tus permisos y puede cambiar los permisos de tus archivos sin que el sistema lo impida.
- **Implementaciones:**
    - **ACL (Access Control Lists):** Descompone la matriz por columnas. Cada objeto tiene una lista de "quién puede entrar".
    - **Capabilities (Capacidades):** Descompone la matriz por filas. El sujeto lleva un "ticket" o token infalsificable que le da acceso al objeto. (Ej: Tokens en OAuth, Handle de archivos).
    - **Permisos Unix:** Simplificación con 3 bits (Read, Write, Execute) para 3 grupos (User, Group, Other).
        - *Conceptos clave:* **Sticky Bit** (solo el dueño puede borrar en `/tmp`), **SetUID** (ejecutar con los permisos del dueño del archivo, no del usuario actual).

3. MAC (Mandatory Access Control) - Control Obligatorio

- **Filosofía:** El **sistema** (administrador) impone las reglas basándose en etiquetas de seguridad. Los usuarios NO pueden cambiar los permisos, ni siquiera sobre sus propios archivos.
- **Modelos Principales (Pregunta de examen):**
    - **Bell-La Padula (Confidencialidad):** Estilo militar.
        - *No Read Up:* Un soldado no puede leer documentos de General.
        - *No Write Down:* Un General no puede escribir secretos en un documento público (para evitar fugas).
    - **Biba (Integridad):** Lo contrario a Bell-La Padula.
        - *No Read Down:* No confíes en información de niveles inferiores.
        - *No Write Up:* Un usuario novato no puede modificar documentos del sistema.

4. RBAC (Role-Based Access Control)

- **Filosofía:** Los permisos no se dan a usuarios, se dan a **Roles** (ej. "Enfermero", "Cajero"). Luego los usuarios se asignan a esos roles.
- **Ventaja:** Simplifica enormemente la gestión en empresas grandes. Si alguien cambia de puesto, solo cambias su rol, no tienes que tocar miles de archivos.

5. ABAC (Attribute-Based Access Control)

- **Filosofía:** El más moderno y flexible. Las reglas usan atributos complejos ("Si es Gerente Y está en la Oficina Y es horario laboral").
- **Uso:** Muy común en servicios Cloud (AWS IAM, Azure) y XACML.

6. Windows Integrity Control (MIC)

Windows implementa un sistema híbrido DAC + MAC.

- Asigna etiquetas de integridad a los procesos: **Low, Medium, High, System**.
- **Regla:** Un proceso de nivel bajo (ej. navegador Web en Internet) no puede escribir en un objeto de nivel alto (ej. archivos de sistema), aunque el usuario tenga permisos DAC. Esto protege contra exploits de navegador.

PDF 11 : SO

1. Arquitectura y Privilegios

El SO actúa como intermediario entre el hardware y las aplicaciones, utilizando diferentes niveles de privilegio para proteger el sistema:

- **Anillos de Ejecución (Protection Rings):**
  - Los procesadores modernos tienen 4 niveles (Rings 0-3).
  - **Ring 0 (Kernel Mode):** Acceso total al hardware e instrucciones privilegiadas. Aquí vive el núcleo del SO.
  - **Ring 3 (User Mode):** Donde corren las aplicaciones normales (navegador, Word). No pueden tocar el hardware directamente.
- **System Calls (Syscalls):** Es la única puerta segura para que una aplicación (Ring 3) pida servicios al Kernel (Ring 0). Se implementan mediante interrupciones software o instrucciones especiales (`traps`).

2. Gestión de Memoria

El hardware (MMU - Memory Management Unit) ayuda al SO a aislar procesos:

- **Memoria Virtual:** Cada proceso cree que tiene toda la memoria para él solo. No puede ver ni tocar la memoria de otros procesos ni del Kernel.
- **Paginación y Segmentación:** Mecanismos para traducir direcciones virtuales a físicas y aplicar permisos (Lectura/Escritura/Ejecución) a bloques de memoria.

3. Vulnerabilidades Críticas: Buffer Overflow

Es el ataque clásico a la gestión de memoria. Ocurre cuando un programa escribe más datos de los que caben en un búfer, sobrescribiendo memoria adyacente (como la dirección de retorno en el Stack).

Mecanismos de Defensa Modernos (¡Muy importante para examen!):

1. **Stack Canaries:** Un valor secreto colocado antes de la dirección de retorno. Si el atacante desborda el búfer, sobrescribe el canario. El sistema comprueba el canario antes de retornar; si cambió, detiene el programa.
2. **NX / DEP (No-Execute / Data Execution Prevention):** Marca zonas de memoria (como el Stack o Heap) como "No Ejecutables". Si un atacante inyecta código allí, el procesador se niega a ejecutarlo.
3. **ASLR (Address Space Layout Randomization):** Aleatoriza las direcciones de memoria donde se cargan las librerías y el stack cada vez que se ejecuta el programa. Hace muy difícil para el atacante saber *dónde* saltar.

4. Tipos de Malware

El PDF clasifica el software malicioso según su comportamiento:

- **Virus:** Necesita un programa anfitrión (host) para replicarse (se "pega" a un .exe).
- **Gusano (Worm):** Se replica por sí mismo a través de la red, sin intervención humana.
- **Troyano:** Parece útil pero esconde una carga maliciosa. No se replica solo.
- **Rootkit:** Malware diseñado para esconderse en el núcleo (Kernel) o reemplazar binarios del sistema para mantener acceso oculto y privilegiado.

5. Virtualización y Contenedores

- **Hypervisores:**
    - **Tipo 1 (Bare Metal):** Corre directo sobre el hardware (ej. ESXi, Hyper-V). Es más eficiente.
    - **Tipo 2 (Hosted):** Corre sobre otro SO (ej. VirtualBox sobre Windows).
- **Contenedores (Docker/LXC):** No virtualizan el hardware, sino el SO. Usan características del Kernel de Linux para aislar procesos:
    - **Namespaces:** Aíslan lo que el proceso *ve* (PIDs, Red, Usuarios).
    - **Cgroups:** Limitan lo que el proceso *usa* (CPU, RAM).

PDF 12 : FIREWALLS

1. Definición y Objetivos

Un firewall no es solo un dispositivo, es una estrategia para interconectar dominios de red distintos (ej. tu red interna segura e Internet inseguro).

- **Función:** Centralizar las políticas de seguridad. Todo el tráfico debe pasar por él (cuello de botella intencional) para aplicar control de acceso, flujo y contenido.
- **Ventaja:** Al centralizar, facilitas la detección de problemas y minimizas el impacto de vulnerabilidades en los equipos internos.

2. Arquitecturas de Defensa

El diseño de la red es tan importante como el aparato en sí:

- **Bastion Host:** Un equipo expuesto a ataques. Debe tener un SO endurecido (hardened), solo servicios esenciales y **nunca** alojar servidores públicos directamente si se puede evitar.
- **DMZ (Zona Desmilitarizada):** Una subred intermedia "sucia" o semipública.
    - Aquí van los servidores públicos (Web, Mail). Si los hackean, no comprometen la red interna.
- **Doble Firewall:** Un firewall externo (permisivo) protege la DMZ y uno interno (restrictivo) protege la red privada. Es la arquitectura más segura.

3. Tipos de Firewalls (Evolución Tecnológica)

Es vital distinguir cómo filtran el tráfico:

- **Filtros de Paquetes (Stateless):**
    - Miran cabeceras IP/TCP individualmente (IP origen, puerto destino). Rápido pero "tonto".
- **Filtros de Estado (Stateful Inspection):**
    - Tienen memoria (contexto). Saben si un paquete pertenece a una conexión ya establecida o si es un fragmento. Son el estándar hoy en día.
- **Pasarelas de Aplicación (Proxies):**
    - Entienden el protocolo (HTTP, FTP). Pueden filtrar comandos específicos (ej. "permitir GET pero denegar POST"). Son más lentos pero más seguros.
- **Pasarelas de Circuito (Circuit Gateways):**
    - Ejemplo: **SOCKS**. Establecen un circuito genérico sin entender el protocolo de aplicación. Requieren modificar el cliente para usar el proxy.

4. Firewalls Personales Software que corre en el PC del usuario (ej. Windows Defender Firewall).

Ventaja: Defensa en profundidad. Protege al equipo si la red falla o si te conectas a un WiFi público inseguro.**Problema:** El usuario promedio no sabe configurarlos y acaba permitiendo todo ("Allow All") para que las apps funcionen.

5. Iptables (Linux)

El estándar en servidores Linux. Se organiza en **Tablas** y **Cadenas** (Chains):**Tablas:** `filter` (decidir si pasa o no), `nat` (cambiar IPs/puertos), `mangle` (modificar paquetes)**Cadenas Clave:**`INPUT`: Tráfico dirigido al propio firewall.`OUTPUT`: Tráfico generado por el firewall.`FORWARD`: Tráfico que atraviesa el firewall (de una red a otra).**Acciones (Verdicts):** `ACCEPT`, `DROP` (silencioso), `REJECT` (avisa al emisor), `LOG`.